

GRAM OPERAÇÕES

Sistema de Ordens de Serviço Civil

DOCUMENTAÇÃO TÉCNICA COMPLETA

Grupo Rede Amazônica
Diretoria de Tecnologia

Índice

1. Visão Geral do Sistema

Objetivo · Funcionalidades · Fluxo Geral

2. Front-End

Telas · Componentes · Navegação · Comunicação com o Backend

3. Back-End

Estrutura · Rotas da API · Autenticação · Upload de Arquivos · E-mail · SSE

4. Banco de Dados

Tabelas · Relacionamentos · Índices · Fluxo de Dados

5. Supabase e Armazenamento

PostgreSQL · Object Storage · Variáveis de Ambiente

6. Fluxo Completo do Sistema

Acesso · Abertura de OS · Registro em Campo · Fechamento · Consulta

7. Estrutura do Projeto

Monorepo · Frontend · Backend · Pacotes Compartilhados

8. Tecnologias Utilizadas

Frontend · Backend · Infraestrutura

9. Integrações e Serviços Externos

Supabase · Object Storage · SMTP · SSE · Canvas API

10. Considerações Técnicas

Pontos Fortes · Boas Práticas · Sugestões Futuras

Este documento descreve o sistema GRAM Operações em linguagem acessível, voltado para qualquer pessoa que precise entender o sistema — seja ela técnica ou não.

1. Visão Geral do Sistema

1.1 Objetivo

O GRAM Operações é um sistema de gestão de Ordens de Serviço (OS) desenvolvido para o Grupo Rede Amazônica. Ele digitaliza e centraliza todo o processo de manutenção civil das unidades do grupo — desde a abertura de uma OS até sua conclusão com assinatura digital — eliminando papéis, planilhas e comunicação informal.

O sistema atende múltiplas unidades regionais (AM, AC, AP, RO, RR, PA) com isolamento de dados por unidade: cada equipe vê apenas as informações da sua região.

1.2 Principais Funcionalidades

Para Gestores

- Criar e atribuir Ordens de Serviço
- Acompanhar status em tempo real
- Dashboard com indicadores e gráficos
- Gerenciar equipe de técnicos (hierarquia)
- Controlar fornecedores e compras
- Agendar serviços em calendário
- Gerenciar arquivos e documentos
- Configurar o sistema (SMTP, alertas)

Para Técnicos e Funcionários

- Registrar OS em campo (celular)
- Fotografar serviços realizados
- Assinar digitalmente ao concluir
- Registrar retirada de materiais
- Registrar PMOC (manutenção preventiva)
- Funciona offline — envia ao reconectar
- Acesso por link compartilhado (sem senha)

1.3 Fluxo Geral de Utilização

Gestor abre a OS

Preenche título, descrição, tipo de serviço, técnico responsável e agenda a data.

Técnico visualiza a OS

Acessa via link compartilhado da sua unidade e verifica o que deve ser feito.

Técnico executa e registra

No campo, usa o celular para fotografar e documentar o serviço realizado.

OS é fechada com assinatura

Técnico ou gestor fecha a OS, coleta assinatura digital do solicitante.

Dados disponíveis em tempo real

Dashboard atualiza; todos os usuários conectados recebem o alerta instantaneamente.

2. Front-End

O front-end é a interface visual que o usuário vê no navegador ou celular. Foi construído como uma SPA (Single Page Application): a página nunca recarrega completamente; apenas o conteúdo muda conforme a navegação.

2.1 Estrutura das Telas

O sistema possui três áreas distintas:

Área	Descrição
Administrativa (com sidebar)	Dashboard, Ordens, Técnicos, Fornecedores, Compras, PMOC, Arquivos, Calendário, Configurações. Exige token de unidade.
Registro (funcionário de campo)	Telas simplificadas: Registrar OS, Fechar OS, PMOC, Materiais. Acessíveis por link direto, sem sidebar.
Pública	Fornecedores e Compras em modo leitura, sem autenticação (link aberto para terceiros).

Páginas Administrativas

Página	Função
Dashboard (/)	Painel com métricas: OS por status, gráficos de evolução mensal, urgências.
Ordens (/ordens)	Lista todas as OS com filtros por status, unidade e período. Abre detalhes ao clicar.
Nova OS (/nova-os)	Formulário completo para o gestor abrir uma nova Ordem de Serviço.
Detalhe da OS (/os/:id)	Exibe todos os dados de uma OS: fotos, status, assinatura digital, histórico.
Técnicos (/tecnicos)	Cadastro e gestão da equipe com hierarquia (gestor → gerente → técnico) e foto.
Indicadores (/indicadores)	Gráficos e métricas de desempenho da equipe e dos serviços executados.
Fornecedores (/fornecedores)	Cadastro completo com CNPJ, endereço, contato e localização (Google Maps).
Compras (/compras)	Planilha de controle de compras e contratações de serviços.
PMOC (/pmoc)	Plano de Manutenção Operacional Contratual — registra manutenções preventivas.
Retirada (/retirada-materiais)	Controle de materiais retirados do almoxarifado pela equipe.
Calendário (/calendario)	Visão mensal de serviços agendados. Alerta automático ao abrir se há OS para o dia seguinte.
Arquivos (/arquivos)	Gerenciador de arquivos e pastas com upload para o Object Storage.
Processos (/processos)	Visualização de processos internos da unidade.
Cadastros (/cadastros)	Gestão de contatos internos da unidade.
Configurações (/configuracoes)	SMTP, modo manutenção, tokens de compartilhamento, alertas.

Páginas de Campo (sem sidebar)

Página	Função
Menu Funcionário (/registrar)	Tela inicial de campo — o funcionário escolhe o tipo de registro.
Registrar OS (/registrar/os)	Funcionário registra uma OS em campo: título, fotos, descrição (formulário simplificado).
Fechar OS (/registrar/fechar-os)	Fecha OS existente: observações, fotos de conclusão, assinatura digital do cliente.
Registrar PMOC (/registrar/pmoc)	Registra itens do plano de manutenção preventiva.
Registrar Materiais (/registrar/materiais)	Registra materiais utilizados em um serviço.

2.2 Organização dos Componentes

- components/ui/ — Design system base: Button, Input, Modal, Card, Toast, Select, Table, etc. Baseados em Radix UI + Tailwind CSS.
- components/layout/ — AppLayout: estrutura base com sidebar de navegação lateral e barra superior com menu de unidade.
- components/tecnicos/ — Lista, formulário e visualização de hierarquia de técnicos.
- components/compras/ — Componentes específicos da planilha de compras.
- components/calendar-alerts.tsx — Alerta de serviços agendados para o dia seguinte (exibido uma vez por sessão).

2.3 Navegação entre Páginas

A navegação usa Wouter — uma biblioteca leve de roteamento similar ao React Router. Todas as rotas de campo (/registrar/*) são acessíveis por link direto, sem necessidade de autenticação. As rotas administrativas exigem um token de unidade válido.

2.4 Comunicação com o Backend

Mecanismo	Quando é usado
TanStack Query + fetch	Para buscar e enviar dados normalmente (GET, POST, PUT, DELETE). Cache de 5 min — evita chamadas repetidas.
Fila offline (localStorage)	Quando o dispositivo está sem internet. Dados salvos localmente e enviados automaticamente ao reconectar.
SSE (Server-Sent Events)	Para receber atualizações em tempo real quando outra pessoa altera o status de uma OS.

O header HTTP X-Share-Token é enviado em todas as requisições para identificar a unidade do usuário. Não há login com usuário e senha.

3. Back-End

O back-end roda em um servidor Node.js e é invisível ao usuário. Ele recebe as requisições do front-end, valida os dados, consulta o banco de dados e retorna as respostas.

3.1 Estrutura

Arquivo	Função
src/index.ts	Ponto de entrada — inicializa o servidor e define a porta (variável PORT).
src/app.ts	Configura o Express: CORS, body parser (limite 50 MB), middlewares, rotas e error handler global.
src/routes/	Uma pasta por recurso: service-orders, technicians, suppliers, storage, dashboard, events, etc.
src/lib/	Utilitários reutilizáveis: logger, share-tokens, SSE broadcast, object storage, system guard.

3.2 Rotas da API

Rota	Métodos	Função
/api/service-orders	GET POST PUT PATCH DELETE	CRUD de OS. GET lista exclui fotos/assinatura por performance. GET/:id retorna tudo.
/api/service-orders/:id/status	PATCH	Altera status e dispara SSE broadcast para todos os clientes.
/api/service-orders/:id/sign	POST	Salva assinatura digital e dados de conclusão.
/api/technicians	GET POST PUT DELETE	CRUD de técnicos com foto e hierarquia de gestão.
/api/suppliers	GET POST PUT DELETE	CRUD de fornecedores.
/api/material-withdrawals	GET POST	Registro e listagem de retiradas de materiais.
/api/purchase-sheets	GET POST PUT	Planilhas de compras por unidade.
/api/pmoc	GET POST PUT	Dados do plano de manutenção preventiva.
/api/file-entries	GET POST PUT DELETE	Gerenciador de arquivos e pastas.
/api/storage/uploads/request-url	POST	Gera URL pré-assinada para upload direto de fotos de técnicos.
/api/storage/uploads/video-url	POST	Gera URL pré-assinada para upload de vídeo.
/api/storage/objects/*	GET	Download de arquivo do Object Storage com validação de

		ACL.
/api/dashboard/summary	GET	Totais de OS por status para o painel.
/api/dashboard/stats	GET	Estatísticas detalhadas por categoria, urgência, período.
/api/events	GET (SSE)	Stream de eventos em tempo real para atualizações de status.
/api/contacts	GET POST PUT DELETE	Cadastro de contatos internos.
/api/settings	GET PUT	Configurações do sistema (SMTP, modo manutenção, tokens).
/api/healthz /health	GET	Verifica saúde do serviço e conectividade real com o banco.

3.3 Autenticação por Token de Unidade

O sistema NÃO usa login com usuário e senha. Em vez disso, utiliza tokens de compartilhamento por unidade — um por região.

Geração do token

O administrador gera tokens em /api/settings. Cada token é HMAC-SHA256(SESSION_SECRET, 'share:UNIDADE'), hex, 48 chars.

Distribuição do link

O link (ex: <https://gram.app?token=abc123...>) é enviado à equipe por WhatsApp ou e-mail.

Uso pelo funcionário

O funcionário clica no link. O token é enviado no header X-Share-Token de toda requisição.

Validação no servidor

Middleware shareTokenMiddleware valida o token com comparação timing-safe e determina a unidade.

Unidade	Nível de Acesso
AM	Administrador — pode ver e operar dados de qualquer unidade.
AC AP RO RR PA	Acesso restrito — vê e opera apenas dados da própria unidade.
Sem token	Modo desenvolvimento — sem restrição (apenas local).

3.4 Upload de Arquivos

Fotos de OS (base64)

- Comprimidas no navegador via Canvas API
- 12 MP (~8 MB) → JPEG 1920px (~250 KB)
- Enviadas no body JSON da requisição
- Salvas na coluna 'photos' (PostgreSQL)
- Limite do servidor: 50 MB por request

Fotos de Técnicos e Docs

- Upload direto ao Object Storage
- Frontend solicita URL pré-assinada
- Arquivo enviado diretamente do browser
- Não passa pelo servidor — mais rápido
- Caminho salvo no banco de dados

3.5 Notificações por E-mail

Nodemailer envia e-mails ao mudar o status de uma OS. As configurações SMTP ficam na tabela settings (não em variáveis de ambiente), permitindo alteração sem reiniciar o servidor. Provedor padrão configurado: Zoho Mail (smtp.zoho.com, porta 465).

3.6 Eventos em Tempo Real (SSE)

Server-Sent Events mantém uma conexão HTTP aberta do servidor para o navegador. Quando qualquer usuário muda o status de uma OS, todos os outros recebem o evento instantaneamente — sem precisar atualizar a página.

- Endpoint: GET /api/events — navegador abre e mantém a conexão
 - Keep-alive ping a cada 25 segundos — evita timeout de proxies e load balancers
 - Reconexão automática com back-off exponencial: 1 s → 2 s → 4 s → máximo 30 s
 - Hook useStatusEvents gerencia a reconexão e expõe o estado (connected / reconnecting / offline) para a UI
-

4. Banco de Dados

O banco é um PostgreSQL hospedado no Supabase. O acesso é feito pelo backend via Drizzle ORM — uma camada TypeScript que traduz operações JavaScript em SQL com segurança de tipos.

4.1 Tabela: service_orders (Ordens de Serviço)

A tabela central do sistema. Cada linha representa uma OS.

Coluna	Tipo	Descrição
id	serial PK	Identificador único auto-incrementado.
number	text	Número da OS (ex: OS-4512). Gerado aleatoriamente na criação.
title	text	Título/descrição curta do serviço.
description	text	Descrição completa do problema ou serviço.
status	text	Estado: aberta em_andamento concluida cancelada.
category	text	Categoria: elétrica, hidráulica, civil, refrigeração, etc.
priority	text	Urgência: baixa media alta.
tipo	text	Tipo: corretiva preventiva preditiva.
formato_servico	text	Formato: civil refrigeracao hidraulica eletrica outros.
technician_id	integer FK	ID do técnico responsável (referência a technicians.id).
technician_name_free	text	Nome livre do técnico quando não está cadastrado no sistema.
unidade	text	Unidade regional: AM, AC, AP, RO, RR, PA.
photos	text	JSON com array de fotos em base64 (comprimidas). Omitido nas listagens por performance.
signature	text	Assinatura digital do cliente (base64 de imagem canvas).
signed_by / signed_at	text / timestamptz	Nome e data/hora de quem assinou.
scheduled_at	timestamptz	Data agendada para o serviço.
completed_at	timestamptz	Data e hora de conclusão.
estimated_value	numeric	Valor estimado do serviço (R\$). Calculado por tabela de referência.
location / department	text	Local do serviço e departamento/setor solicitante.
notes	text	Observações adicionais.
origem	text	Como a OS foi criada: manual importada.
created_at / updated_at	timestamptz	Data de criação e última atualização (auto-gerenciados).

4.2 Tabela: technicians (Técnicos)

Coluna	Tipo	Descrição
id	serial PK	Identificador único.
name	text	Nome completo.
specialty	text	Especialidade (ex: Elétrica, Civil, Refrigeração).
position	text	Cargo (ex: Técnico, Gerente, Diretor).
area	text	Área de atuação.
phone / email	text	Contatos.
manager_id	integer FK	ID do supervisor direto (auto-referência). Cria hierarquia organizacional.
photo_url	text	Caminho da foto no Object Storage.
unidade	text	Unidade regional onde atua.
is_corporate	boolean	Cargo corporativo – aparece em todas as unidades (ex: Diretor de Tecnologia).
active	boolean	Se o técnico está ativo no sistema.

4.3 Demais Tabelas

Tabela	Finalidade
suppliers	Fornecedores: razão social, CNPJ/CPF, endereço, cidade, UF, contato, e-mail, link Google Maps.
contacts	Contatos internos da unidade: nome, telefone, departamento, e-mail.
material_withdrawals	Retiradas de materiais: quem retirou, data, tipo, lista de itens em JSONB.
file_entries	Arquivos e pastas: name, is_folder, parent_id (auto-ref. para estrutura de diretórios), storage_path.
purchase_sheets	Planilhas de compras: uma por unidade, dados em JSONB. Chave única (unidade).
pmoc_data	Dados do PMOC: chave primária storage_key, dados em JSONB. Estrutura flexível por unidade.
settings	Configurações do sistema: pares chave/valor (SMTP host, porta, usuário, senha, modo manutenção).
links	Links úteis cadastrados por unidade (documentos, referências).

4.4 Relacionamentos

- service_orders.technician_id ↔ technicians.id: cada OS está vinculada a um técnico cadastrado.
- technicians.manager_id ↔ technicians.id: auto-relacionamento que cria a hierarquia da equipe (técnico ↔ gerente ↔ diretor).
- file_entries.parent_id ↔ file_entries.id: auto-relacionamento que cria a estrutura de pastas e subpastas.

4.5 Índices de Performance

Criados automaticamente pela migração para garantir respostas rápidas mesmo com muitos registros:

Índice	Finalidade
idx_so_unidade_created	Listagem de OS por unidade ordenada por data — a query mais frequente do sistema.
idx_so_unidade_status	Filtro de OS por unidade e status (ex: OS abertas da unidade AM).
idx_so_status	Filtro global por status (para métricas do dashboard).
idx_so_technician_id	OS de um técnico específico.
idx_so_scheduled_at	OS agendadas por data (calendário).
idx_tech_manager_id	Busca da equipe de um gestor (hierarquia).
idx_fe_parent_id	Conteúdo de uma pasta no gerenciador de arquivos.
idx_mw_unidade / idx_mw_created_at	Listagem de retiradas de materiais por unidade e data.

4.6 Fluxo de Leitura e Gravação

Frontend faz requisição

Ex: buscar lista de OS abertas da unidade AM.

Backend valida

Valida token de unidade e parâmetros da requisição via Zod.

Drizzle ORM gera SQL

Monta query com filtros, colunas específicas (sem photos/signature na listagem) e índices.

PostgreSQL executa

Banco executa via pool de conexões; índices composto aceleram a busca.

Backend serializa e retorna

JSON formatado com nomes de técnicos enriquecidos (join eficiente por inArray).

Frontend cacheia

TanStack Query armazena por 5 min — reutiliza sem nova requisição até expirar.

5. Supabase e Armazenamento

5.1 Supabase — Banco de Dados PostgreSQL

Supabase é uma plataforma de banco de dados PostgreSQL gerenciado na nuvem. O projeto usa o Supabase exclusivamente como banco de dados — não usa a autenticação nem o storage do Supabase.

- O host direto do Supabase usa IPv6, mas o servidor de hospedagem só tem IPv4.
- A conexão é feita via URL do pooler do Supabase (porta 6543) — compatível com IPv4.
- O script migrate.mjs usa `dns.setDefaultResultOrder('ipv6first')` para resolver esse conflito.

5.2 Gerenciamento do Schema (scripts/migrate.mjs)

O schema é gerenciado por um script Node.js personalizado — não pelo Drizzle Kit. O ambiente de hospedagem não tem TTY, impedindo o uso interativo do drizzle-kit push. O script é idempotente: pode ser executado múltiplas vezes sem problemas (usa `ADD COLUMN IF NOT EXISTS` e `CREATE TABLE IF NOT EXISTS` em todas as operações).

5.3 Object Storage (Armazenamento de Arquivos)

Para arquivos maiores (fotos de técnicos, documentos, vídeos), o sistema usa o Replit Object Storage — baseado no Google Cloud Storage, similar ao Amazon S3.

Tipo de Arquivo	Armazenamento
Fotos de OS	Base64 na coluna 'photos' do PostgreSQL (comprimidas pelo browser via Canvas API).
Fotos de Técnicos	Object Storage — URL pré-assinada; upload direto do browser.
Documentos e Arquivos	Object Storage — estrutura de pastas em <code>file_entries</code> .
Vídeos de OS	Object Storage — URL pré-assinada; upload direto do browser.

5.4 Variáveis de Ambiente

Variável	Uso
DATABASE_URL	URL de conexão ao pooler do Supabase (inclui SSL).
SESSION_SECRET	Segredo HMAC para gerar e validar tokens de compartilhamento.
DEFAULT_OBJECT_STORAGE_BUCKET_ID	ID do bucket do Replit Object Storage.
PRIVATE_OBJECT_DIR	Diretório para objetos privados no bucket.
PUBLIC_OBJECT_SEARCH_PATHS	Caminhos públicos no bucket (separados por vírgula).
PORT	Porta do servidor (atribuída automaticamente pela plataforma).
BASE_PATH	Caminho base da aplicação (ex: /os-civil).

PGHOST PGPORT PGUSER PGPASSWORD PGDATABASE	Credenciais diretas ao PostgreSQL (usadas por scripts de migração).
---	---

6. Fluxo Completo do Sistema

6.1 Acesso ao Sistema

Usuário recebe o link

Ex: <https://gram.app/os-civil?token=abc123...> — enviado pelo administrador via WhatsApp ou e-mail.

Token é validado

Backend valida o HMAC e identifica a unidade (AM, AC, AP, RO, RR ou PA).

Sistema carrega

Frontend detecta a unidade, configura o contexto e exibe o dashboard ou menu de campo.

Cache inicial

TanStack Query carrega OS recentes, técnicos e configurações em paralelo.

SSE conecta

Conexão SSE abre automaticamente para receber atualizações em tempo real.

6.2 Abertura de OS pelo Gestor

Gestor acessa Nova OS (/nova-os)

Formulário completo com todos os campos da OS.

Preenche os dados

Título, descrição, categoria, prioridade, técnico responsável, data agendada, local, departamento.

Adiciona fotos (opcional)

Galeria ou câmera. Fotos comprimidas automaticamente: 12 MP → ~250 KB.

Confirma e envia

POST /api/service-orders — OS salva no banco com status 'aberta', número gerado automaticamente.

Notificações disparadas

E-mail ao técnico responsável (se SMTP configurado) + SSE broadcast para todos os usuários.

6.3 Registro pelo Funcionário de Campo

Funcionário acessa /registrar

Via link compartilhado no celular — sem login necessário.

Escolhe 'Registrar OS'

Menu simples com as opções disponíveis para o funcionário de campo.

Preenche formulário simplificado

Título, tipo de serviço, descrição, fotos com câmera do celular (Android/iOS).

Sem internet?

Dados salvos na fila offline (localStorage). Indicador visual de itens pendentes.

Ao reconectar

Fila drena automaticamente. Itens já enviados não são reenviados (proteção anti-duplicidade).

- Baseada em localStorage — funciona sem Service Worker ou instalação.
- Drena automaticamente: ao reconectar, ao voltar à aba, ou ao abrir o sistema.

- IDs de itens já enviados são guardados — page reload não causa reenvio.
- Sincronizada entre múltiplas abas abertas simultaneamente.
- Timeout de 15s por item — um request travado não bloqueia os demais.

6.4 Fechamento da OS

Acessa /registrar/fechar-os

Busca a OS pelo número ou seleciona da lista de OS abertas.

Documenta a conclusão

Preenche observações, adiciona fotos do serviço concluído.

Coleta assinatura digital

Cliente assina na tela (touch ou mouse). Assinatura capturada como imagem base64.

Envia fechamento

PATCH /api/service-orders/:id/status → status muda para 'concluida', completed_at preenchido.

Dashboard e alertas

Métricas do painel atualizam. SSE notifica todos os usuários conectados.

6.5 Consulta e Relatórios

- Gestor acessa /ordens → lista completa com filtros por status, unidade, período, técnico.
 - Clica em uma OS → os/:id — todos os dados, fotos em galeria e assinatura digital visualizável.
 - Dashboard (/) exibe totais, gráficos de evolução mensal, OS urgentes destacadas.
 - Calendário (/calendario) mostra OS agendadas em formato mensal com alerta proativo.
 - Indicadores (/indicadores) exibe gráficos de performance por técnico, categoria e período.
-

7. Estrutura do Projeto

7.1 Monorepo com pnpm Workspaces

O projeto é um monorepo — um único repositório com múltiplos pacotes. Isso permite compartilhar código (tipos, schemas, cliente de banco) entre o frontend e o backend sem duplicação.

Pacote / Pasta	Função
artifacts/os-civil/	Frontend — aplicação React/Vite servida ao usuário no navegador.
artifacts/api-server/	Backend — servidor Express.js com todas as rotas da API.
lib/api-client-react/	Hooks React Query e funções fetch tipadas — compartilhados entre páginas do frontend.
lib/api-zod/	Schemas Zod de validação — usados por frontend E backend (fonte única de verdade dos tipos).
lib/db/	Pool de conexão PostgreSQL e schema Drizzle ORM — compartilhado pelo backend.
scripts/migrate.mjs	Script de migração do banco — executado no deploy para atualizar o schema.

7.2 Frontend — artifacts/os-civil/src/

Pasta	Função
pages/	Uma página por tela do sistema (dashboard.tsx, nova-os.tsx, registrar-os.tsx, etc.).
components/ui/	Design system base: Button, Input, Modal, Card, Toast, Select, Table, Badge, etc.
components/layout/	AppLayout: estrutura da página com sidebar, topbar e navegação.
hooks/	Hooks customizados: useOfflineQueue, useStatusEvents, useOnlineStatus, useSystemStatus.
lib/	Utilitários: media-utils (compressão de fotos), offline-queue-storage, configuração do API client.
contexts/	Contextos React: UnitContext (unidade ativa), ShareContext (token), StatusAlertsContext.
App.tsx	Roteador principal — define todas as rotas e instancia os providers globais.
index.css	Estilos globais e variáveis de cor do design system (Tailwind CSS custom tokens).

7.3 Backend — artifacts/api-server/src/

Arquivo	Função
routes/service-orders.ts	Listagem, criação, atualização, mudança de status e assinatura de OS.
routes/technicians.ts	CRUD de técnicos com hierarquia e foto no Object Storage.

routes/storage.ts	URLs pré-assinadas para upload e download de arquivos.
routes/dashboard.ts	Queries agregadas (single-pass SQL) para métricas do painel.
routes/events.ts	Endpoint SSE com keep-alive e limpeza automática de clientes desconectados.
routes/settings.ts	Configurações do sistema e geração de tokens de compartilhamento.
lib/share-tokens.ts	Geração e validação de tokens HMAC-SHA256 com comparação timing-safe.
lib/sse-broadcast.ts	Registro de clientes SSE e fan-out de eventos para todos os conectados.
lib/objectStorage.ts	Interface com Google Cloud Storage via Replit Sidecar proxy.
lib/system-guard.ts	Middleware de modo manutenção e restrição requireAMUnit.
lib/logger.ts	Logger Pino estruturado em JSON com serializers para req/res.

8. Tecnologias Utilizadas

8.1 Frontend

Tecnologia	Função no Projeto
React 19	Biblioteca de interfaces. Gerencia o que aparece na tela e como muda com os dados.
TypeScript	JavaScript com tipos estáticos. Previne erros antes de executar — todo o código é tipado.
Vite	Ferramenta de build e servidor de desenvolvimento. Compila TypeScript e empacota para produção.
Tailwind CSS	Framework de estilos utilitários. Estilização declarativa diretamente no JSX.
Radix UI	Componentes base acessíveis (Modal, Select, Dropdown, Tooltip). Prontos e compatíveis com WCAG.
TanStack Query	Gerencia chamadas à API: cache automático, estados de loading/error, revalidação inteligente.
Wouter	Roteador de páginas leve. Controla qual página exibir conforme a URL.
Zod	Validação de formulários e schemas. Garante que dados estão no formato correto antes de enviar.
React Hook Form	Gerenciamento eficiente de formulários — sem re-renderização excessiva.
Lucide React	Biblioteca de ícones SVG com mais de 1000 ícones consistentes.
Framer Motion	Animações declarativas para transições suaves de UI.
Recharts	Gráficos declarativos para o dashboard e indicadores.
Capacitor	Empacota a aplicação web como app nativo iOS/Android.
date-fns	Utilitários de datas: formatação, comparação, cálculo de períodos.
Canvas API (nativa)	Compressão de fotos de câmera no browser antes do upload.

8.2 Backend

Tecnologia	Função no Projeto
Node.js 24	Runtime JavaScript do servidor. Executa todo o código backend.
Express.js 5	Framework web. Define rotas, middlewares e o ciclo completo de requisição/resposta.
Drizzle ORM	ORM TypeScript. Permite escrever queries SQL com segurança de tipos.
PostgreSQL	Banco de dados relacional. Armazena todos os dados do sistema.
Zod	Validação de entrada nas rotas do backend — rejeita dados inválidos antes de tocar o banco.
Pino + pino-http	Logger de alta performance em JSON com serializers para req/res.
Nodemailer	Envio de e-mails via SMTP (configuração armazenada no banco).

@google-cloud/storage	SDK para upload/download no Replit Object Storage (GCS).
bcryptjs	Hash de senhas (módulo preparado para autenticação futura).
cors	Middleware CORS — permite requisições do frontend (domínio diferente).
esbuild	Compilador e bundler do backend TypeScript. Extremamente rápido.
pg (node-postgres)	Driver PostgreSQL. Usado pelo pool de conexões e pelos scripts de migração.

8.3 Infraestrutura

Serviço	Função
Replit	Plataforma de hospedagem. Gerencia processos, variáveis de ambiente, domínio e deploy.
Supabase	Hospedagem gerenciada do PostgreSQL. Backups automáticos, conexão via pooler.
Replit Object Storage	Armazenamento de arquivos (fotos de técnicos, documentos, vídeos). Baseado em GCS.
pnpm Workspaces	Gerenciador de pacotes e monorepo. Deduplica dependências entre os pacotes.

9. Integrações e Serviços Externos

9.1 Supabase — PostgreSQL

- Tipo: Banco de dados PostgreSQL gerenciado na nuvem.
- Conexão: Via URL do pooler (DATABASE_URL) — compatibilidade IPv4/IPv6.
- ORM: Drizzle ORM gerencia as queries no backend com segurança de tipos.
- Schema: Gerenciado por scripts/migrate.mjs (idempotente, sem necessidade de TTY).
- NÃO utiliza: Autenticação Supabase nem Supabase Storage.

9.2 Replit Object Storage

- Tipo: Armazenamento de objetos (API compatível com Google Cloud Storage).
- Uso: Fotos de técnicos, documentos da unidade, vídeos de OS.
- Acesso: URLs pré-assinadas temporárias geradas pelo backend — upload direto do browser.
- Download: Via /api/storage/objects/* com validação de ACL (público/privado).
- Configuração: Variáveis DEFAULT_OBJECT_STORAGE_BUCKET_ID, PRIVATE_OBJECT_DIR, PUBLIC_OBJECT_SEARCH_PATHS.

9.3 SMTP — Notificações por E-mail

- Biblioteca: Nodemailer (Node.js).
- Configuração: Armazenada na tabela settings — alterável sem reiniciar o servidor.
- Provedor configurado: Zoho Mail (smtp.zoho.com, porta 465, SSL).
- Gatilho: Mudança de status de OS (abertura, atribuição).

9.4 SSE — Eventos em Tempo Real

- Protocolo: Server-Sent Events (HTTP unidirecional — servidor envia ao cliente).
- Endpoint: GET /api/events — navegador mantém a conexão aberta.
- Fan-out: Um broadcast() envia o evento JSON para todos os clientes registrados.
- Reconexão automática no frontend com back-off exponencial (1 s a 30 s).
- Não requer WebSocket — solução simples e confiável para este caso de uso.

9.5 Canvas API — Compressão de Imagens

- API nativa do navegador — sem dependência externa.
 - Comprime fotos de câmera de ~8 MB para ~250 KB antes do upload.
 - Redimensiona para máximo 1920 px na maior dimensão.
 - Recodifica para JPEG com qualidade 0.82.
 - Fallback para FileReader quando o Canvas não está disponível (dispositivos antigos).
 - Tratamento especial para Android/iOS: câmeras Samsung entregam file.type vazio — bypass do filtro MIME.
-

10. Considerações Técnicas

10.1 Pontos Fortes da Arquitetura

Ponto Forte	Detalhamento
Offline-first para campo	Funcionários registram OS sem internet. Fila local com proteção anti-duplicidade, timeout por item e sincronização entre abas.
Tipos compartilhados	Schemas Zod definidos em lib/api-zod/ usados por frontend e backend. Impossível ter inconsistência de tipos entre os dois lados.
Performance nas listagens	Coluna photos excluída da query de listagem (base64 pesado). Apenas GET/:id busca as fotos. Evita payloads de dezenas de MB na tela principal.
Segurança por HMAC	Tokens determinísticos sem estado — sem banco necessário para validação. Comparação timing-safe previne ataques de timing.
SSE para tempo real	Solução simples e eficaz para atualizações em tempo real sem a complexidade de WebSockets ou polling.
Compressão de fotos	Resolve o principal problema mobile: câmeras 12 MP geravam uploads de 8–20 MB. Agora ~250 KB por foto — uploads instantâneos em 4G.
Multi-unidade com isolamento	Uma instância atende todas as unidades. Isolamento garantido pelo token em todas as queries SQL.
Health checks reais	Endpoint /api/healthz verifica conectividade efetiva com o banco — não apenas que o processo está rodando.
Error handler global	Express 5 captura qualquer erro não tratado nas rotas — nenhuma exceção derruba o servidor.
Migração idempotente	scripts/migrate.mjs pode ser executado múltiplas vezes sem risco — seguro para rodar no startup de todo deploy.

10.2 Boas Práticas Identificadas

- Validação de entrada com Zod em todas as rotas do backend — dados inválidos rejeitados antes de tocar o banco.
- Logger estruturado (Pino) em JSON — facilita análise de logs em produção e integração com ferramentas de observabilidade.
- Índices de banco criados automaticamente na migração — performance garantida desde o início, sem ação manual.
- TanStack Query com staleTime de 5 minutos e gcTime de 15 minutos — reduz chamadas desnecessárias à API.
- Lazy loading de todas as páginas — bundle inicial menor, carregamento mais rápido no primeiro acesso.
- Todas as credenciais e segredos em variáveis de ambiente — nenhuma senha ou chave no código-fonte.
- CORS configurado explicitamente com lista de métodos e headers permitidos.
- Payload máximo de 50 MB no body parser — suporta fotos em base64 sem rejeitar requests legítimos.
- Timeout de 15 s por item na fila offline — um request travado não bloqueia os demais.
- Enrich de técnicos por inArray (não full table scan) — queries eficientes mesmo com grande volume de OS.

10.3 Sugestões Futuras de Melhoria

Observações técnicas — não devem ser implementadas sem avaliação e priorização pela equipe.

Sugestão	Justificativa
Mover fotos de OS para Object Storage	Fotos em base64 no banco crescem indefinidamente. Object Storage é mais escalável e reduz custo e tamanho do banco.
Autenticação com usuário e senha	Tokens de compartilhamento são práticos mas qualquer portador do link tem acesso. Login individual permitiria auditoria nominal.
Service Worker para PWA completo	A fila offline usa localStorage. Um Service Worker interceptaria requisições de rede para funcionamento offline mais robusto.
Paginação no backend	A listagem retorna todas as OS da unidade. Com grande volume, isso pode ficar lento. Cursor-based pagination limitaria a N itens.
Testes automatizados abrangentes	O sistema cresce continuamente. Testes unitários para lógica de negócio e E2E para fluxos críticos evitariam regressões.
Exportação em PDF/Excel	Gestores frequentemente precisam compartilhar relatórios. Exportação nativa evitaria capturas de tela e planilhas manuais.
Rate limiting nas rotas públicas	Rotas sem autenticação (fornecedores públicos, compras) ficam expostas a abuso. Um rate limiter por IP seria prudente.